

Лабораторная работа III

Реализация masked FlashAttention на Triton

Касьянов А.В.

4 июня 2026 г.

Содержание

1	Введение	3
2	Постановка задачи	3
3	Математические основы FlashAttention	4
3.1	Online softmax	4
3.2	Исключение полностью замаскированных блоков	5
4	Математическое описание обратного распространения	5
5	Описание реализации src/backend/flash_attention.py	6
5.1	Общая структура файла	6
5.2	torch_causal_attention	6
5.3	_flash_attn_fwd_kernel	7
5.4	_flash_attention_forward	7
5.5	_flash_attn_bwd_kernel	8
5.6	_flash_attention_backward	8
5.7	FlashCausalAttentionFunction	9
5.8	flash_causal_attention и FlashCausalAttention	9
6	Тестирование корректности	9
7	Методика сравнительного анализа	9
7.1	Источник данных	9
7.2	Измерение времени	10
7.3	Измерение памяти	10
8	Результаты экспериментов	10
8.1	Параметры эксперимента	10
8.2	Сравнение времени выполнения	11
8.3	Сравнение пикового потребления памяти	11
9	Анализ результатов	12
9.1	Поведение времени выполнения	12
9.2	Поведение потребления памяти	13
10	Почему Triton-реализация оказалась медленнее	13
10.1	PyTorch использует высокооптимизированные backend-операции	13
10.2	Учебный характер реализации	14
10.3	Накладные расходы online softmax	14
10.4	Маски и ветвления	14
10.5	Неоптимальные размеры блоков	14
10.6	Использование float32 в промежуточных вычислениях	15
10.7	Стоимость atomic add в backward	15
10.8	Повторное вычисление вероятностей в backward	15
10.9	Малые и средние размеры задачи	15
10.10	Итоговое объяснение замедления	16
11	Выводы	16

1 Введение

Целью данной лабораторной работы является реализация прямого и обратного распространения для маскированного механизма внимания типа FlashAttention с использованием языка Triton, а также экспериментальное сравнение полученной реализации с эталонной реализацией на PyTorch.

Механизм внимания является центральной вычислительной операцией в трансформерных архитектурах. Для входных тензоров запросов, ключей и значений

$$Q, K, V \in \mathbb{R}^{B \times H \times N \times D}, \quad (1)$$

где B — размер батча, H — число голов внимания, N — длина последовательности, D — размерность одной головы, стандартный scaled dot-product attention определяется выражением

$$O = \text{Softmax} \left(\frac{QK^\top}{\sqrt{D}} \right) V. \quad (2)$$

В авторегрессионных моделях используется causal mask, запрещающая позиции i обращаться к будущим позициям $j > i$. В этом случае матрица оценок внимания задаётся как

$$\tilde{S}_{ij} = \begin{cases} \frac{q_i k_j^\top}{\sqrt{D}}, & j \leq i, \\ -\infty, & j > i. \end{cases} \quad (3)$$

После этого вероятности внимания вычисляются по строкам:

$$P_{ij} = \frac{\exp(\tilde{S}_{ij})}{\sum_{t=0}^{N-1} \exp(\tilde{S}_{it})}. \quad (4)$$

Выход attention равен

$$O_i = \sum_{j=0}^{N-1} P_{ij} V_j. \quad (5)$$

Наивная реализация требует материализации матриц $\tilde{S}$ и P размера $N \times N$ для каждой пары батча и головы. Поэтому асимптотическая сложность по памяти равна

$$\mathcal{M}_{\text{naive}} = O(BHN^2). \quad (6)$$

FlashAttention уменьшает потребление памяти за счёт блочной обработки матриц Q , K , V и применения online softmax, благодаря чему полная матрица внимания не сохраняется в глобальной памяти GPU.

2 Постановка задачи

В рамках лабораторной работы требуется реализовать:

- 1) kernel прямого распространения masked FlashAttention;
- 2) исключение из вычислений полностью замаскированных блоков;
- 3) kernel обратного распространения masked FlashAttention;

- 4) Python-wrapper для forward и backward;
- 5) класс, наследуемый от `torch.autograd.Function`;
- 6) модуль `torch.nn.Module`;
- 7) тестирование корректности прямого и обратного распространения;
- 8) сравнительный анализ времени выполнения и пикового потребления памяти.

В работе используется causal mask, определённая в формуле (3).

3 Математические основы FlashAttention

3.1 Online softmax

Ключевая идея FlashAttention состоит в том, что softmax можно вычислять поблочно без хранения полной матрицы $\tilde{S}$.

Пусть фиксирована строка i . Для неё в процессе обработки блоков поддерживаются три величины:

$$m_i, \quad \ell_i, \quad a_i. \quad (7)$$

Здесь m_i — текущий максимум обработанных элементов строки, ℓ_i — сумма экспонент относительно текущего максимума, a_i — ненормированный аккумулятор выхода.

Пусть на очередном шаге обрабатывается блок оценок $S_i^{(t)}$. Тогда новый максимум вычисляется как

$$m_i^{\text{new}} = \max \left(m_i, \max_j S_{ij}^{(t)} \right). \quad (8)$$

Для перехода от старого максимума к новому вводится множитель перенормировки:

$$\alpha_i = \exp(m_i - m_i^{\text{new}}). \quad (9)$$

Экспоненты текущего блока относительно нового максимума равны

$$p_{ij}^{(t)} = \exp \left(S_{ij}^{(t)} - m_i^{\text{new}} \right). \quad (10)$$

Сумма экспонент обновляется по формуле

$$\ell_i^{\text{new}} = \alpha_i \ell_i + \sum_j p_{ij}^{(t)}. \quad (11)$$

Аккумулятор результата обновляется следующим образом:

$$a_i^{\text{new}} = \alpha_i a_i + \sum_j p_{ij}^{(t)} V_j. \quad (12)$$

После обработки всех блоков выходная строка attention вычисляется как

$$O_i = \frac{a_i}{\ell_i}. \quad (13)$$

Для обратного распространения дополнительно сохраняется величина log-sum-exp:

$$\text{LSE}_i = \log \sum_j \exp \left(\tilde{S}_{ij} \right). \quad (14)$$

Через online softmax она выражается как

$$\text{LSE}_i = m_i + \log \ell_i. \quad (15)$$

Формулы (8)–(13) позволяют получить тот же результат, что и наивное вычисление (2), но без хранения полной матрицы attention.

3.2 Исключение полностью замаскированных блоков

Рассмотрим блок строк Q с индексами

$$i \in \{r \cdot B_M, r \cdot B_M + 1, \dots, r \cdot B_M + B_M - 1\}, \quad (16)$$

где $B_M = \text{BLOCK_M}$.

Для causal mask элемент S_{ij} разрешён только при условии

$$j \leq i. \quad (17)$$

Если очередной блок ключей и значений начинается с позиции `start_n`, то блок полностью замаскирован, когда все его столбцы строго больше всех строк текущего блока. Поэтому достаточно проверять условие

$$\text{start_n} < (r + 1)B_M. \quad (18)$$

Именно условие (18) используется в Triton-кernels для исключения полностью замаскированных блоков из вычислений.

4 Математическое описание обратного распространения

Пусть задан градиент функции потерь по выходу attention:

$$dO = \frac{\partial L}{\partial O}. \quad (19)$$

Поскольку

$$O = PV, \quad (20)$$

градиент по V равен

$$dV = P^\top dO. \quad (21)$$

Градиент по матрице вероятностей P равен

$$dP = dOV^\top. \quad (22)$$

Для softmax используется стандартная формула дифференцирования:

$$dS_{ij} = P_{ij} \left(dP_{ij} - \sum_t P_{it} dP_{it} \right). \quad (23)$$

Так как

$$O_i = \sum_t P_{it} V_t, \quad (24)$$

то

$$\sum_t P_{it} dP_{it} = \sum_t P_{it} (dO_i V_t^\top) = dO_i \left(\sum_t P_{it} V_t \right)^\top = dO_i O_i^\top. \quad (25)$$

Введём величину

$$D_i = \sum_c dO_{ic} O_{ic}. \quad (26)$$

Тогда формула (23) принимает вид

$$dS_{ij} = P_{ij} (dP_{ij} - D_i). \quad (27)$$

Градиенты по Q и K вычисляются как

$$dQ = dSK \cdot \frac{1}{\sqrt{D}}, \quad (28)$$

$$dK = dS^\top Q \cdot \frac{1}{\sqrt{D}}. \quad (29)$$

Матрица P в backward не хранится, а восстанавливается по блокам с использованием сохранённого LSE:

$$P_{ij} = \exp \left(\tilde{S}_{ij} - \text{LSE}_i \right). \quad (30)$$

Таким образом, формулы (21)–(30) задают математическую основу реализованного backward-кernels.

5 Описание реализации src/backend/flash_attention.py

5.1 Общая структура файла

В файле src/backend/flash_attention.py реализованы следующие компоненты:

- 1) torch_causal_attention;
- 2) _flash_attn_fwd_kernel;
- 3) _flash_attention_forward;
- 4) _flash_attn_bwd_kernel;
- 5) _flash_attention_backward;
- 6) FlashCausalAttentionFunction;
- 7) flash_causal_attention;
- 8) FlashCausalAttention.

5.2 torch_causal_attention

Функция torch_causal_attention является эталонной реализацией на PyTorch. Она используется для проверки корректности Triton-реализации.

В ней явно вычисляется матрица оценок attention:

$$S = \frac{QK^\top}{\sqrt{D}}. \quad (31)$$

Затем применяется нижнетреугольная causal mask из формулы (3), после чего выполняются softmax и умножение на V в соответствии с формулами (4) и (5).

Основной фрагмент функции имеет вид:

```
scores = torch.matmul(q.float(), k.float().transpose(-2, -1)) * scale
causal_mask = torch.tril(torch.ones((n, n), device=q.device, dtype=torch.bool))
scores = scores.masked_fill(~causal_mask[None, None, :, :], float("-inf"))
attn = torch.softmax(scores, dim=-1)
out = torch.matmul(attn, v.float())
```

Недостаток данной функции состоит в том, что она материализует матрицы размера $B \times H \times N \times N$, что соответствует оценке памяти (6).

5.3 _flash_attn_fwd_kernel

Функция `_flash_attn_fwd_kernel` является Triton-кernels-м прямого распространения.

Каждый экземпляр Triton-программы обрабатывает один блок строк Q и одну пару “батч–голова”. Сетка запуска имеет вид

$$\text{grid} = \left(\left\lceil \frac{N}{B_M} \right\rceil, B \cdot H \right), \quad (32)$$

где $B_M = \text{BLOCK_M}$.

Индексы программы:

$$\text{pid_m} = \text{tl.program_id}(\text{axis}=0), \quad \text{pid_bh} = \text{tl.program_id}(\text{axis}=1). \quad (33)$$

Индекс батча и головы восстанавливается следующим образом:

$$b = \left\lfloor \frac{\text{pid_bh}}{H} \right\rfloor, \quad h = \text{pid_bh} - bH. \quad (34)$$

Оффсеты строк, столбцов и размерности головы задаются формулами

$$\text{offs_m} = \text{pid_m}B_M + \{0, 1, \dots, B_M - 1\}, \quad (35)$$

$$\text{offs_n} = \{0, 1, \dots, B_N - 1\}, \quad (36)$$

$$\text{offs_d} = \{0, 1, \dots, B_D - 1\}. \quad (37)$$

Здесь $B_N = \text{BLOCK_N}$, $B_D = \text{BLOCK_D}$.

Адрес элемента тензора в памяти вычисляется через страйды:

$$\text{addr}(b, h, n, d) = \text{base} + b \cdot s_b + h \cdot s_h + n \cdot s_n + d \cdot s_d. \quad (38)$$

Именно поэтому в kernel передаются страйды для Q , K , V и O .

Для предотвращения выхода за границы используются маски:

$$\text{mask_m} = \text{offs_m} < N, \quad \text{mask_d} = \text{offs_d} < D. \quad (39)$$

Causal mask внутри блока имеет вид

$$\text{causal_mask}_{ij} = [\text{cur_n}_j \leq \text{offs_m}_i]. \quad (40)$$

Полностью замаскированные блоки не вычисляются благодаря условию (18). Это является важным требованием задания.

Внутри kernel реализуются формулы online softmax (8)–(13). Кроме того, сохраняется тензор LSE по формуле (15), необходимый для backward.

5.4 _flash_attention_forward

Функция `_flash_attention_forward` является Python-wrapper для прямого Triton-кernels-а. Она выполняет следующие действия:

- 1) проверяет размерности входных тензоров;
- 2) проверяет, что тензоры находятся на CUDA;
- 3) приводит Q , K , V к contiguous-представлению;

- 4) выделяет выходной тензор O ;
- 5) выделяет тензор $\text{LSE} \in \mathbb{R}^{B \times H \times N}$;
- 6) вычисляет параметры сетки (32);
- 7) запускает `_flash_attn_fwd_kernel`.

Тензор LSE имеет форму

$$\text{LSE} \in \mathbb{R}^{B \times H \times N}. \quad (41)$$

Он сохраняется, поскольку в backward вероятности P восстанавливаются по формуле (30).

5.5 `_flash_attn_bwd_kernel`

Функция `_flash_attn_bwd_kernel` реализует обратное распространение.

Каждый экземпляр программы обрабатывает блок строк Q и соответствующий блок dO . Для каждого блока K, V заново вычисляется блок scores :

$$S_{ij}^{(t)} = \frac{q_i k_j^\top}{\sqrt{D}}. \quad (42)$$

После применения `causal mask` восстанавливается блок вероятностей:

$$P_{ij}^{(t)} = \exp \left(\tilde{S}_{ij}^{(t)} - \text{LSE}_i \right). \quad (43)$$

Далее вычисляются градиенты согласно формулам (21), (22), (27), (28) и (29).

Величина D_i из формулы (26) вычисляется в коде как:

```
d_i = tl.sum(o.to(tl.float32) * do.to(tl.float32), axis=1)
```

Градиент dQ записывается обычной операцией `store`, поскольку каждый блок строк Q обрабатывается ровно одним `program instance`. Однако dK и dV получают вклады от разных блоков Q , поэтому для них используются атомарные операции:

```
tl.atomic_add(..., dk, ...)
tl.atomic_add(..., dv, ...)
```

Атомарное накопление необходимо для корректности, но оно является одной из причин снижения производительности backward-кernels.

5.6 `_flash_attention_backward`

Функция `_flash_attention_backward` является wrapper для backward-кernels. Она выделяет тензоры dQ , dK и dV .

При этом

$$dK_0 = 0, \quad dV_0 = 0, \quad (44)$$

так как dK и dV накапливаются через `atomic add`.

5.7 FlashCausalAttentionFunction

Класс `FlashCausalAttentionFunction` наследуется от `torch.autograd.Function`. Он необходим, поскольку PyTorch не знает, как автоматически дифференцировать пользовательские Triton-кernels.

Метод `forward` вызывает `_flash_attention_forward` и сохраняет для backward тензоры

$$Q, K, V, O, \text{LSE}. \quad (45)$$

Метод `backward` принимает dO и возвращает

$$dQ, dK, dV. \quad (46)$$

Для параметров `block_m` и `block_n` возвращается `None`, поскольку они не являются тензорами и не требуют градиента.

5.8 flash_causal_attention и FlashCausalAttention

Функция `flash_causal_attention` является пользовательским API:

```
def flash_causal_attention(q, k, v, block_m=64, block_n=64):  
    return FlashCausalAttentionFunction.apply(q, k, v, block_m, block_n)
```

Класс `FlashCausalAttention` является модулем PyTorch и позволяет использовать реализацию внутри моделей, наследуемых от `torch.nn.Module`.

6 Тестирование корректности

Корректность forward проверялась сравнением:

$$O_{\text{torch}} \approx O_{\text{triton}}. \quad (47)$$

Для проверки backward сравнивались градиенты:

$$dQ_{\text{torch}} \approx dQ_{\text{triton}}, \quad dK_{\text{torch}} \approx dK_{\text{triton}}, \quad dV_{\text{torch}} \approx dV_{\text{triton}}. \quad (48)$$

В качестве функции потерь использовалось скалярное произведение выхода attention и случайного тензора G :

$$L = \sum_{b,h,i,d} O_{b,h,i,d} G_{b,h,i,d}. \quad (49)$$

Такой тест проверяет общий векторно-якобиановый продукт, вычисляемый механизмом `autograd`.

7 Методика сравнительного анализа

7.1 Источник данных

Данные для сравнительного анализа не брались из внешнего датасета. Они генерировались синтетически с помощью функции `torch.randn`:

$$Q, K, V \sim \mathcal{N}(0, 1). \quad (50)$$

Использование синтетических данных является корректным для данного микробенчмарка, поскольку стоимость dense attention определяется прежде всего размерами тензоров:

$$B, H, N, D, \quad (51)$$

а не конкретными числовыми значениями внутри Q , K и V .

Маска causal attention также не зависит от значений данных и задаётся только индексами i и j в соответствии с условием (17).

7.2 Измерение времени

Время выполнения измерялось с помощью функции `triton.testing.do_bench`. Перед измерением выполнялась синхронизация CUDA:

```
torch.cuda.synchronize()
```

Синхронизация необходима, поскольку CUDA-операции асинхронны относительно CPU. Ускорение рассчитывалось по формуле

$$\text{speedup} = \frac{T_{\text{torch}}}{T_{\text{triton}}}. \quad (52)$$

Также использовалась величина замедления Triton относительно PyTorch:

$$\text{triton_slower_x} = \frac{T_{\text{triton}}}{T_{\text{torch}}}. \quad (53)$$

Из формул (52) и (53) следует:

$$\text{triton_slower_x} = \frac{1}{\text{speedup}}. \quad (54)$$

7.3 Измерение памяти

Пиковое потребление памяти измерялось средствами PyTorch CUDA allocator:

```
torch.cuda.reset_peak_memory_stats()
torch.cuda.max_memory_allocated()
```

Отношение потребления памяти вычислялось как

$$\text{memory_ratio} = \frac{M_{\text{torch}}}{M_{\text{triton}}}. \quad (55)$$

Если $\text{memory_ratio} > 1$, то Triton-реализация потребляет меньше памяти.

8 Результаты экспериментов

8.1 Параметры эксперимента

Во всех экспериментах использовались следующие параметры:

$$B = 1, \quad H = 8, \quad D = 64, \quad \text{dtype} = \text{float16}. \quad (56)$$

Длины последовательности:

$$N \in \{128, 256, 512, 1024, 2048\}. \quad (57)$$

Для Triton-реализации использовались параметры:

$$\text{BLOCK_M} = 64, \quad \text{BLOCK_N} = 64. \quad (58)$$

8.2 Сравнение времени выполнения

В таблице 1 приведены результаты сравнения времени выполнения PyTorch и Triton. Значения `speedup` и `triton_slower_x` вычислены по формулам (52) и (53).

Таблица 1: Сравнение времени выполнения и памяти для PyTorch и Triton

seq_len	batch	heads	head_dim	dtype	torch_ms	triton_ms	speedup	torch_mem	triton_mem	mem_ratio	triton_slower_x
128	1	8	64	float16	0.149011	0.305591	0.487616	23.656250	22.269531	1.062270	2.050796
256	1	8	64	float16	0.177980	0.696053	0.255699	27.578125	22.773438	1.210978	3.910851
512	1	8	64	float16	0.507444	2.522369	0.201178	41.515625	23.781250	1.745729	4.970731
1024	1	8	64	float16	1.828495	10.081272	0.181375	93.765625	25.796875	3.634767	5.513426
2048	1	8	64	float16	7.191472	39.425503	0.182407	295.765625	29.828125	9.915663	5.482258

Из таблицы 1 видно, что во всех рассмотренных экспериментах величина `speedup` меньше единицы. Следовательно, по формуле (52) Triton-реализация оказалась медленнее эталонной PyTorch-реализации.

Для наглядности результаты времени выполнения представлены на рисунке 1.

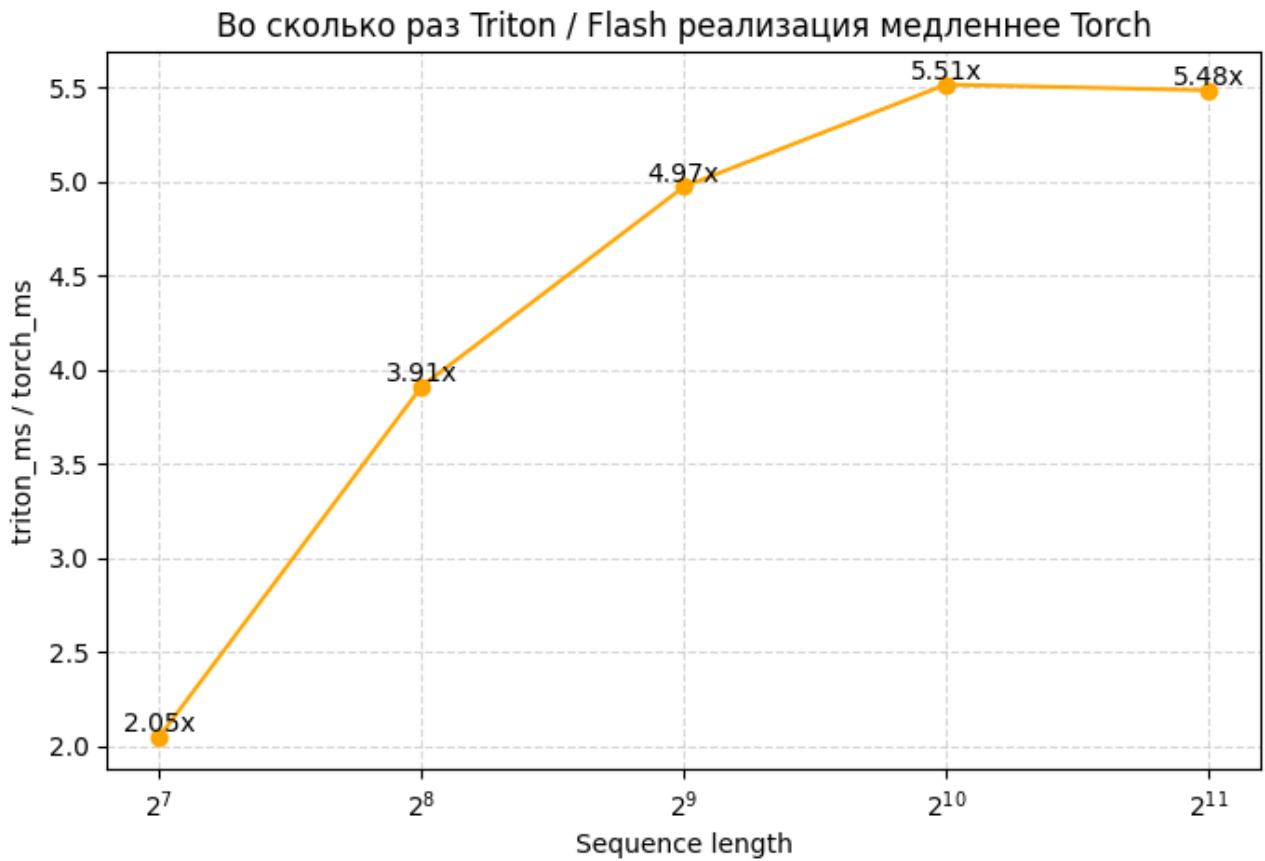


Рис. 1: Сравнение времени выполнения PyTorch и Triton

8.3 Сравнение пикового потребления памяти

В таблице 2 приведены результаты сравнения пикового потребления памяти. Отношение памяти вычислялось по формуле (55).

Таблица 2: Сравнение пикового потребления памяти

seq_len	batch	heads	head_dim	dtype	torch_peak_mem_mb	triton_peak_mem_mb	memory_ratio
128	1	8	64	float16	23.656250	22.269531	1.062270
256	1	8	64	float16	27.578125	22.773438	1.210978
512	1	8	64	float16	41.515625	23.781250	1.745729
1024	1	8	64	float16	93.765625	25.796875	3.634767
2048	1	8	64	float16	295.765625	29.828125	9.915663

Результаты из таблицы 2 показывают, что Triton-реализация потребляет существенно меньше памяти при больших длинах последовательности. При $N = 2048$ отношение памяти равно 9.915663, то есть PyTorch-реализация потребляет почти в десять раз больше памяти.

Графическое представление результатов приведено на рисунке 2.

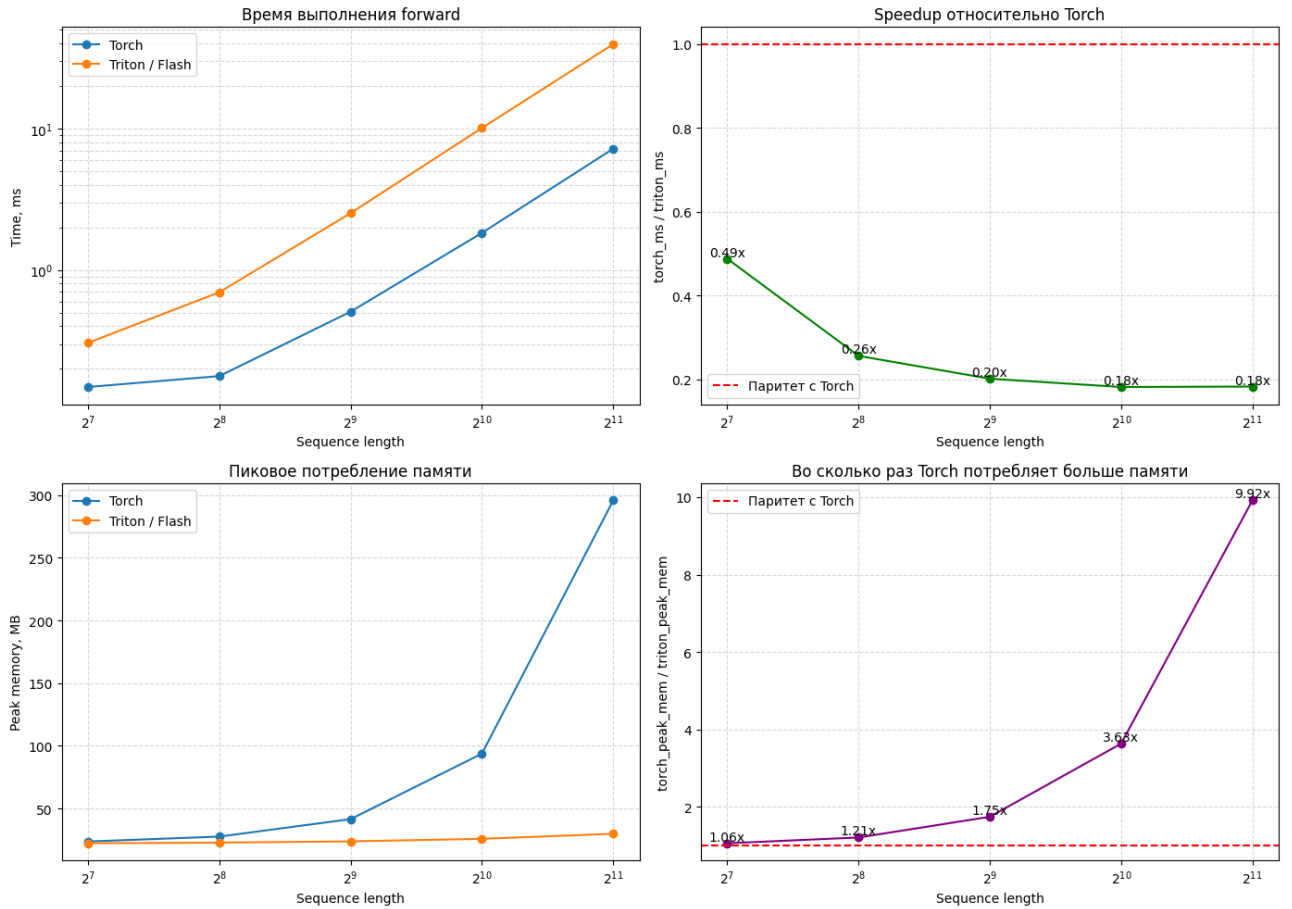


Рис. 2: Сравнение пикового потребления памяти PyTorch и Triton

9 Анализ результатов

9.1 Поведение времени выполнения

Из таблицы 1 следует, что время выполнения Triton-реализации растёт быстрее, чем время выполнения PyTorch-реализации. Например, при $N = 128$ имеем:

$$T_{\text{torch}} = 0.149011 \text{ ms}, \quad T_{\text{triton}} = 0.305591 \text{ ms}. \quad (59)$$

В этом случае Triton медленнее в

$$\frac{0.305591}{0.149011} \approx 2.050796 \quad (60)$$

раза.

При $N = 2048$ имеем:

$$T_{\text{torch}} = 7.191472 \text{ ms}, \quad T_{\text{triton}} = 39.425503 \text{ ms}. \quad (61)$$

В этом случае Triton медленнее в

$$\frac{39.425503}{7.191472} \approx 5.482258 \quad (62)$$

раза.

Таким образом, экспериментально наблюдается не ускорение, а замедление. Это не противоречит теории FlashAttention, поскольку реализованный kernel является учебным, а сравнение проводится с высокооптимизированными операциями PyTorch.

9.2 Поведение потребления памяти

В отличие от времени выполнения, результаты по памяти демонстрируют преимущество Triton-реализации. При $N = 2048$:

$$M_{\text{torch}} = 295.765625 \text{ MB}, \quad M_{\text{triton}} = 29.828125 \text{ MB}. \quad (63)$$

Отношение памяти:

$$\frac{295.765625}{29.828125} \approx 9.915663. \quad (64)$$

Это соответствует основной идее FlashAttention: не хранить полную матрицу attention размера $N \times N$. Для PyTorch-реализации память растёт приблизительно согласно оценке (6), тогда как для FlashAttention основной дополнительный тензор имеет размер (41).

10 Почему Triton-реализация оказалась медленнее

10.1 PyTorch использует высокооптимизированные backend-операции

Наивная функция `torch_causal_attention` выглядит простой, однако операции

```
torch.matmul  
torch.softmax  
torch.matmul
```

на GPU исполняются не как Python-код. Они вызывают высокооптимизированные CUDA-библиотеки и kernel-реализации, включая cuBLAS, специализированные softmax-kernels и в некоторых конфигурациях оптимизированные реализации scaled dot-product attention.

Матричные умножения в PyTorch для `float16` обычно используют Tensor Cores. Поэтому операции вида

$$QK^T \text{ и } PV \quad (65)$$

выполняются чрезвычайно эффективно.

Учебный Triton-кernels не использует весь набор оптимизаций, доступных в промышленных библиотеках.

10.2 Учебный характер реализации

Реализованный kernel предназначен для демонстрации принципа FlashAttention, а не для достижения промышленной производительности. В нём отсутствуют:

- 1) autotuning размеров блоков;
- 2) отдельные kernel-варианты для разных D ;
- 3) оптимизация числа warp'ов под конкретную GPU;
- 4) тонкая оптимизация использования регистров;
- 5) оптимизация доступа к shared memory;
- 6) специальные стратегии backward без интенсивного использования atomic add;
- 7) split-K и split-Q стратегии.

Следовательно, сравнение в таблице 1 фактически является сравнением PyTorch backend с учебной реализацией на Triton.

10.3 Накладные расходы online softmax

В PyTorch attention вычисляется через крупные высокооптимизированные матричные операции. В Triton-реализации для каждого блока дополнительно выполняются операции online softmax:

$$m_i^{\text{new}}, \quad \alpha_i, \quad \ell_i^{\text{new}}, \quad a_i^{\text{new}}, \quad (66)$$

определённые формулами (8)–(12).

Эти операции повышают численную устойчивость и позволяют экономить память, но создают дополнительные арифметические и управляющие расходы.

10.4 Маски и ветвления

Causal attention требует проверки условия (17). Внутри блока это реализуется через логические маски вида (40). Кроме того, используется условие пропуска полностью замаскированных блоков (18).

Хотя пропуск блоков уменьшает число вычислений, сами маски и ветвления увеличивают сложность kernel. Для достижения высокой производительности такие проверки должны быть тщательно согласованы со структурой тайлинга, что в учебной реализации сделано лишь базовым образом.

10.5 Неоптимальные размеры блоков

В экспериментах использовались параметры (58). Однако оптимальные значения BLOCK_M и BLOCK_N зависят от:

- 1) архитектуры GPU;
- 2) размера D ;
- 3) длины последовательности N ;
- 4) числа голов H ;

- 5) типа данных;
- 6) числа регистров на поток;
- 7) оверсапу;
- 8) пропускной способности памяти.

Без autotuning выбранные параметры могут приводить к низкой оверсапу или избыточному давлению на регистры. Это напрямую увеличивает T_{triton} в формуле (52).

10.6 Использование float32 в промежуточных вычислениях

Для численной устойчивости многие промежуточные величины вычисляются в float32:

$$m_i, \ell_i, a_i, dS, dQ, dK, dV. \quad (67)$$

Это полезно для точности, но может снижать производительность по сравнению с агрессивным использованием float16 и Tensor Cores. PyTorch/cuBLAS для матричных умножений часто достигает более эффективного баланса между float16 входами и float32 накоплением.

10.7 Стоимость atomic add в backward

В backward-кernels градиенты dK и dV накапливаются через атомарные операции. Это связано с тем, что один и тот же блок K, V используется несколькими блоками Q .

Математически накопление соответствует формулам (21) и (29). Однако на уровне GPU несколько program instances могут одновременно записывать в один и тот же адрес. Для корректности используется atomic add.

Атомарная операция дороже обычной записи, поскольку должна разрешать конфликты параллельной записи. Это особенно существенно для больших N , что частично объясняет рост triton_slower_x в таблице 1.

10.8 Повторное вычисление вероятностей в backward

FlashAttention экономит память, не сохраняя матрицу P . Однако в backward она восстанавливается по формуле (30). Для этого заново вычисляются scores:

$$S = \frac{QK^\top}{\sqrt{D}}. \quad (68)$$

Таким образом, FlashAttention обменивает память на дополнительные вычисления. Для больших моделей это часто выгодно, поскольку глобальная память является узким местом. Но в данной учебной реализации дополнительные вычисления и накладные расходы kernel оказываются достаточно велики, чтобы сделать Triton медленнее PyTorch.

10.9 Малые и средние размеры задачи

Для $N = 128$ и $N = 256$ матрица attention сравнительно мала. Поэтому PyTorch успевает эффективно выполнить материализацию scores и softmax, используя оптимизированные backend-kernels.

При таких размерах преимущество FlashAttention по памяти ещё не полностью конвертируется в преимущество по времени. Это видно из первых строк таблицы 1, где при $N = 128$ memory ratio равен только 1.062270, а Triton уже медленнее примерно в 2.05 раза.

10.10 Итоговое объяснение замедления

Итак, Triton-реализация оказалась медленнее PyTorch по следующим причинам:

- 1) PyTorch использует высокооптимизированные CUDA-библиотеки.
- 2) Матричные умножения PyTorch эффективно используют Tensor Cores.
- 3) Triton-кernels имеют учебный характер и не autotuned.
- 4) Online softmax добавляет дополнительные операции.
- 5) Causal mask требует дополнительных логических масок.
- 6) Backward использует дорогостоящие atomic add.
- 7) Вероятности attention в backward пересчитываются заново.
- 8) Выбранные размеры блоков не обязательно оптимальны для конкретной GPU.

При этом результаты по памяти в таблице 2 подтверждают основное преимущество FlashAttention: отказ от хранения полной матрицы attention приводит к существенной экономии памяти.

11 Выводы

В ходе лабораторной работы была реализована учебная версия masked FlashAttention на Triton. Реализация включает прямое распространение, обратное распространение, поддержку causal mask, wrapper-функции, интеграцию с `torch.autograd.Function` и модуль `torch.nn.Module`.

Математическая корректность реализации основана на online softmax, описанном формулами (8)–(13), и на формулах обратного распространения (21)–(29).

Экспериментальные результаты показали, что учебная Triton-реализация оказалась медленнее PyTorch-реализации по времени выполнения. Например, при $N = 2048$ Triton медленнее примерно в 5.48 раза, что следует из формулы (62) и таблицы 1.

Однако по потреблению памяти Triton-реализация показала существенное преимущество. При $N = 2048$ PyTorch потребляет почти в десять раз больше памяти, что отражено в формуле (64) и таблице 2.

Следовательно, реализованный kernel демонстрирует основную идею FlashAttention: уменьшение потребления памяти за счёт отказа от материализации матрицы attention. Замедление по времени объясняется не ошибочностью подхода, а учебным характером реализации, отсутствием autotuning, использованием atomic add и сравнением с крайне оптимизированными backend-операциями PyTorch.